Mathematical Modeling and Greedy Optimization: Base Conversions, Binary Exponentiation, and Two-Pointer Algorithms for Linear Search Space Reduction

Abstract

This paper presents a formal mathematical analysis and algorithmic treatment of four classic computer science problems at the intersection of number theory, bijective arithmetic, and two-pointer greedy optimization. First, we examine the Excel Column Title problem, modeling it as a bijective base-26 positional numeral system without a zero element, and develop its inverse, Excel Column Number. Second, we analyze Binary Exponentiation (modular power), proving its logarithmic time complexity $O(\log N)$ via recursive halving and binary bit representation. Third, we address the Container With Most Water problem, formalizing its search space and proving the correctness of the two-pointer greedy strategy via linear search space reduction. Fourth, we examine the Three Sum problem, showing how sorting combined with a two-pointer search reduces the time complexity from cubic $O(N^3)$ to quadratic $O(N^2)$, and formalize the duplicate-avoidance invariants. For each problem, we present complete, production-ready Java code templates, rigorous proofs of correctness, and detailed asymptotic time and space complexity analyses.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis

1 Introduction

Algorithmic optimization frequently requires combining mathematical properties with greedy pointer manipulations to reduce the computational search space of a problem. In technical evaluations, competitive programming (CP), and rigorous academic curricula such as the Graduate Aptitude Test in Engineering (GATE), problems that seem to demand brute-force enumerations are optimized to linear or logarithmic execution times through first-principles analysis.

This paper presents an exhaustive, graduate-level formalization of four primary algorithms that lie at the intersection of number theory, base arithmetic, and the two-pointer technique. Rather than treating these techniques as heuristics, we model their operations mathematically and prove their correctness using greedy search space pruning and logarithmic reduction.

The structure of this paper is organized as follows:

1) Bijective Base-26 conversions (§2): A rigorous analysis of positional numeral systems without a zero digit, formalizing the algorithms and providing complete Java implementations for converting integers to Excel column titles and vice-versa.
2) Logarithmic Power Computation (§3): An investigation of modular binary exponentiation, proving its $\mathcal{O}(\log N)$ complexity and providing iterative and recursive templates.
3) Linear Search Space Reduction via Two Pointers (§4): A mathematical proof of the correctness of the greedy pointer convergence strategy for the "Container With Most Water" problem, demonstrating how $\mathcal{O}(N^2)$ candidate states are pruned to $\mathcal{O}(N)$.
4) Sorting-Assisted Target Searching (§5): A formalization of the "Three Sum" problem, demonstrating how sorting enables a quadratic-time two-pointer search with invariant-based duplicate avoidance.
5) Exhaustive Problem Walkthroughs (§6): Detailed execution traces, math steps, and verification of edge cases for each of the four domains.

2 Bijective Base-26 Positional Numeral Systems

Standard positional numeral systems of base B map integers using digits from the set $\{0, 1, \ldots, B-1\}$. However, the system utilized for Excel column titles represents a bijective base-B system (specifically, $B = 26$), which does not contain a symbol representing zero. Instead, the digit set is $\{1, 2, \ldots, 26\}$, mapped to the characters $\{A, B, \ldots, Z\}$.

2.1 Mathematical Model of Excel Column Conversion

In a standard base-26 system with digits $\{0, \ldots, 25\}$, a string $S = s_{k-1}s_{k-2}\ldots s_0$ evaluates to:

$$V_{\text{standard}}(S) = \sum_{i=0}^{k-1} s_i \cdot 26^i \quad \text{where } s_i \in \{0, \ldots, 25\} \tag{1}$$

In the bijective base-26 system used for Excel columns, the value of the string is computed as:

$$V_{\text{bijective}}(S) = \sum_{i=0}^{k-1} \text{val}(s_i) \cdot 26^i \quad \text{where val}(s_i) \in \{1, \ldots, 26\} \tag{2}$$

Here, $\text{val}(A) = 1, \text{val}(B) = 2, \ldots, \text{val}(Z) = 26$. This bijection guarantees that every positive integer maps to a unique, non-empty string, eliminating the ambiguity of leading zeros (e.g., in a standard base system, A and AA would evaluate to the same value if $A = 0$, but here $\text{val}(A) = 1$ while $\text{val}(AA) = 1 \cdot 26^1 + 1 \cdot 26^0 = 27$).

2.2 Algorithm 1: Integer to Excel Title

To convert a positive integer A to its bijective base-26 string representation, we cannot perform standard modulo division directly. In standard division, $A \pmod{26}$ yields a remainder in $\{0, \ldots, 25\}$. Because our digits represent $\{1, \ldots, 26\}$, we shift the integer by subtracting 1 before performing the division. This maps the range $[1, 26]$ to $[0, 25]$:

$$\text{rem} = (A - 1) \pmod{26} \tag{3}$$

$$A_{\text{next}} = \lfloor (A - 1)/26 \rfloor \tag{4}$$

The character corresponding to rem (where $0 \to A, 1 \to B, \ldots, 25 \to Z$) is prepended to the result, and the process repeats until $A = 0$.

2.2.1 Java Reference Implementation

```
public class ExcelColumnTitle {
    public static String convertToTitle(int A) {
        StringBuilder sb = new StringBuilder();
        while (A > 0) {
            A--; // Shift to 0-indexed range [0, 25]
            int rem = A % 26;
            sb.append((char) ('A' + rem));
            A /= 26;
        }
        return sb.reverse().toString();
    }
}
```

- Time Complexity: In each step, the integer is divided by 26. The number of iterations is proportional to the number of digits in the output string:

$$T(A) = \mathcal{O}(\log_{26} A) \tag{5}$$

- Space Complexity: The only memory used is the 'StringBuilder' to accumulate the characters. The length of the string is:

$$S(A) = \mathcal{O}(\log_{26} A) \tag{6}$$

2.3 Algorithm 2: Excel Title to Integer

The inverse operation converts a string of uppercase letters back to a decimal integer. We scan the string from left to right. At each character s_i, we multiply our accumulated value by 26 and add the numeric value of the character:

$$V_0 = 0 \tag{7}$$

$$V_{i+1} = V_i \cdot 26 + (\text{character } s_i -' A' + 1) \tag{8}$$

2.3.1 Java Reference Implementation

```
public class ExcelColumnNumber {
    public static int titleToNumber(String A) {
        int result = 0;
        for (int i = 0; i < A.length(); i++) {
            result = result * 26 + (A.charAt(i) - 'A' + 1);
        }
        return result;
    }
}
```

- Time Complexity: We iterate through the string exactly once. For a string of length L:

$$T(L) = \mathcal{O}(L) = \mathcal{O}(\log_{26} A) \tag{9}$$

- Space Complexity: The algorithm uses a single integer accumulator, requiring:

$$S(L) = \mathcal{O}(1) \tag{10}$$

3 Logarithmic Power Computation (Binary Exponentiation)

Computing the power of a base $x^n \pmod M$ is a fundamental operation in modular arithmetic, primality testing, and cryptography. A naive approach of multiplying x by itself n times requires $\mathcal{O}(n)$ multiplications. Binary Exponentiation utilizes the binary representation of the exponent to reduce this complexity to $\mathcal{O}(\log n)$.

3.1 Mathematical Formulation

The recursive property of binary exponentiation is formulated as:

$$x^n = \begin{cases} 1 & \text{if } n = 0 \\ \left(x^{n/2}\right)^2 & \text{if } n \text{ is even} \\ x \cdot \left(x^{(n-1)/2}\right)^2 & \text{if } n \text{ is odd} \end{cases} \tag{11}$$

By applying this property, the exponent is halved at each step, bounding the total number of multiplication operations by $2\lfloor \log_2 n \rfloor$.

3.2 Java Reference Implementations

We present both the recursive (top-down) and iterative (bottom-up) implementations of modular binary exponentiation.

3.2.1 Iterative Formulation

The iterative version processes the bits of the exponent n from least significant to most significant. If the current bit of n is 1, we multiply our result by the current base. We then square the base and shift n right by 1 bit.

```
public class BinaryExponentiation {
    public static long power(long x, long n, long M) {
        if (n == 0) return 1 % M;
        long res = 1;
        x = x % M; // Handle cases where x >= M
        if (x < 0) x += M; // Handle negative base cases

        while (n > 0) {
            if ((n & 1) == 1) {
                res = (res * x) % M;
            }
            x = (x * x) % M;
            n >>= 1;
        }
        return res;
    }
}
```

3.2.2 Recursive Formulation

```
public class BinaryExponentiationRecursive {
    public static long power(long x, long n, long M) {
        if (n == 0) return 1 % M;
        x = x % M;
        if (x < 0) x += M;

        long half = power(x, n / 2, M);
        long halfSq = (half * half) % M;

        if (n % 2 == 0) {
            return halfSq;
        } else {
            return (halfSq * x) % M;
        }
    }
}
```

3.3 Complexity Analysis

- Time Complexity: At each step of the loop (or recursive call), the exponent n is divided by 2 (via 'n »= 1' or 'n / 2'). The loop terminates when $n = 0$. The total number of multiplications is bounded by $2 \log_2 n$:

$$T(n) = \mathcal{O}(\log n) \tag{12}$$

- Space Complexity: The iterative version utilizes a constant number of primitive variables, yielding $S(n) = \mathcal{O}(1)$. The recursive version requires stack frames proportional to the recursion depth, yielding $S(n) = \mathcal{O}(\log n)$.

4 Linear Search Space Reduction via Two Pointers

The "Container With Most Water" problem presents a classic optimization task. Given an array A of N non-negative integers representing vertical wall heights, we seek to find two indices i and j ($0 \leq i < j < N$) that maximize the volume of water enclosed:

$$V(i, j) = (j - i) \cdot \min(A[i], A[j]) \tag{13}$$

A naive search of all pairs (i, j) requires $\mathcal{O}(N^2)$ evaluations. By using a two-pointer approach, we reduce the search space to $\mathcal{O}(N)$.

4.1 Formal Proof of Correctness (Linear Search Space Reduction)

We initialize two pointers, i at 0 (the left boundary) and j at $N - 1$ (the right boundary). We calculate the current volume $V(i, j)$. To update the pointers, we compare the heights $A[i]$ and $A[j]$, and move the pointer pointing to the smaller height inward. We present the mathematical proof showing why this greedy pruning is correct and safe.

Theorem 1. Let i and j be the current left and right pointer boundaries respectively, with $i < j$. If $A[i] \leq A[j]$, then for any intermediate index k such that $i < k < j$, the volume $V(i, k)$ is strictly bounded by $V(i, j)$:

$$V(i, k) < V(i, j) \tag{14}$$

Consequently, the index i can be safely discarded (i.e., we increment $i \leftarrow i + 1$) without missing the global maximum.

Proof. Let k be any index satisfying $i < k < j$. By definition, the volume of the container formed by boundaries i and k is:

$$V(i, k) = (k - i) \cdot \min(A[i], A[k]) \tag{15}$$

Since k lies strictly to the left of j, the width of this container satisfies:

$$(k - i) < (j - i) \tag{16}$$

Now consider the height of this container, $\min(A[i], A[k])$. By definition of the minimum function:

$$\min(A[i], A[k]) \leq A[i] \tag{17}$$

Multiplying these two inequalities:

$$(k - i) \cdot \min(A[i], A[k]) < (j - i) \cdot A[i] \tag{18}$$

Because we assumed $A[i] \leq A[j]$, the height of the original container at boundaries i and j is:

$$\min(A[i], A[j]) = A[i] \tag{19}$$

Substituting this back into the right side of the inequality:

$$(k-i)\cdot\min(A[i],A[k]) < (j-i)\cdot\min(A[i],A[j]) \tag{20}$$

Which simplifies to:

$$V(i,k) < V(i,j) \tag{21}$$

This proves that no container starting at boundary i and ending at any intermediate boundary k can ever exceed the volume of the container formed by boundaries i and j. Thus, the state space $\{(i,k) \mid i<k<j\}$ contains no candidates that could be the global maximum. The boundary i can be safely discarded by incrementing $i \leftarrow i+1$.

By symmetry, if $A[j] < A[i]$, then for any intermediate index k such that $i<k<j$, the volume satisfies $V(k,j) < V(i,j)$. Thus, the right boundary j can be safely discarded by decrementing $j \leftarrow j-1$. □

4.2 Java Reference Implementation

```
import java.util.ArrayList;

public class ContainerWithMostWater {
   public static int maxArea(ArrayList<Integer> A) {
      if (A == null || A.size() < 2) return 0;

      int maxVal = 0;
      int i = 0;
      int j = A.size() - 1;

      while (i < j) {
         int width = j - i;
         int height = Math.min(A.get(i), A.get(j));
         int area = width * height;

         maxVal = Math.max(maxVal, area);

         // Greedy pointer movement based on our theorem
         if (A.get(i) <= A.get(j)) {
            i++;
         } else {
            j--;
         }
      }
      return maxVal;
   }
}
```

4.3 Complexity Analysis

- Time Complexity: In each iteration of the loop, the distance between the two pointers $j-i$ decreases by exactly 1 (either i increments or j decrements). The loop terminates when $i=j$. The number of iterations is exactly $N-1$. In each step, we perform constant-time arithmetic and comparisons. Therefore:

$$T(N) = \mathcal{O}(N) \tag{22}$$

- Space Complexity: The algorithm only utilizes a few primitive pointer and area variables. No auxiliary data structures are allocated:

$$S(N) = \mathcal{O}(1) \tag{23}$$

5 Sorting-Assisted Target Searching (Three Sum)

The "Three Sum" problem requires us to find all unique triplets $[A[i], A[j], A[k]]$ in an unsorted array of integers such that their sum is exactly zero:

$$A[i]+A[j]+A[k]=0 \quad \text{where } i<j<k \tag{24}$$

A naive cubic search takes $\mathcal{O}(N^3)$ time and generates duplicates. By sorting the array and combining it with a two-pointer search, we can solve this in $\mathcal{O}(N^2)$ time while avoiding duplicates.

5.1 Invariant-Based Duplicate Avoidance

To ensure that the triplets in our output are unique, we establish mathematical invariants during the search:

1) Outer Loop Invariant: Let the sorted array be A. We fix the first element $A[i]$. If $i > 0$ and $A[i] = A[i-1]$, we skip this index. This prevents processing duplicate triplets starting with the same value.
2) Two-Pointer Invariants: Let the two pointers be $L = i+1$ and $R = N-1$. If we find a triplet such that $A[i] + A[L] + A[R] = 0$, we record it. Then, to find other potential triplets with the same fixed $A[i]$, we must move both pointers inward. To prevent duplicates, we advance L and decrement R past any elements equal to their neighbors:

$$\text{while } A[L] = A[L-1], \quad L \leftarrow L+1 \tag{25}$$

$$\text{while } A[R] = A[R+1], \quad R \leftarrow R-1 \tag{26}$$

5.2 Java Reference Implementation

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class ThreeSum {
   public static List<List<Integer>> threeSum(int[] A) {
      List<List<Integer>> result = new ArrayList<>();
      if (A == null || A.length < 3) return result;

      Arrays.sort(A); // Sort the array to enable two-pointer search
      int n = A.length;

      for (int i = 0; i < n - 2; i++) {
         // Avoid duplicates for the first element of the triplet
         if (i > 0 && A[i] == A[i - 1]) continue;

         int left = i + 1;
         int right = n - 1;

         while (left < right) {
            int sum = A[i] + A[left] + A[right];

            if (sum == 0) {
               result.add(Arrays.asList(A[i], A[left], A[right]));
               left++;
               right--;

               // Avoid duplicates for the second element
               while (left < right && A[left] == A[left - 1]) left++;
               // Avoid duplicates for the third element
               while (left < right && A[right] == A[right + 1]) right--;

            } else if (sum < 0) {
               left++; // Sum is too small, move left pointer to increase sum
            } else {
               right--; // Sum is too large, move right pointer to decrease sum
            }
         }
      }
      return result;
   }
}
```

5.3 Complexity Analysis

- Time Complexity: Sorting the array of size N takes $\mathcal{O}(N \log N)$ time. The outer loop runs $N-2$ times. In each iteration, the inner two-pointer search runs in $\mathcal{O}(N)$ because $left$ and $right$ start at opposite ends and meet in the

middle. The total time for the search phase is:

$$T_{\text{search}}(N) = \sum_{i=0}^{N-3} \mathcal{O}(N-i) = \mathcal{O}(N^2) \tag{27}$$

Combining the sorting and search phases:

$$T(N) = \mathcal{O}(N \log N) + \mathcal{O}(N^2) = \mathcal{O}(N^2) \tag{28}$$

- Space Complexity: The two-pointer search uses $\mathcal{O}(1)$ auxiliary space. The space complexity is determined by the sorting algorithm, which typically requires $\mathcal{O}(\log N)$ or $\mathcal{O}(N)$ memory. Thus:

$$S(N) = \mathcal{O}(\log N) \quad \text{to } \mathcal{O}(N) \tag{29}$$

6 Exhaustive Problem Walkthroughs

In this section, we walk through detailed, step-by-step executions of the algorithms.

6.1 Problem 1: Excel Column conversions

Task: Find the string representation of $A = 702$ and verify by converting it back to an integer.

Execution Trace (Integer to Title):

1) Iteration 1:

$$A = 702 \tag{30}$$
$$A_{\text{shifted}} = 701 \tag{31}$$
$$\text{rem} = 701 \pmod{26} = 25 \quad (\text{character } 'Z') \tag{32}$$
$$A_{\text{next}} = \lfloor 701/26 \rfloor = 26 \tag{33}$$

2) Iteration 2:

$$A = 26 \tag{34}$$
$$A_{\text{shifted}} = 25 \tag{35}$$
$$\text{rem} = 25 \pmod{26} = 25 \quad (\text{character } 'Z') \tag{36}$$
$$A_{\text{next}} = \lfloor 25/26 \rfloor = 0 \tag{37}$$

The loop terminates since $A = 0$. Reversing our collected characters yields: "ZZ".

Execution Trace (Title to Integer):

$$A_{\text{string}} = \text{"ZZ"} \tag{38}$$
$$V_0 = 0 \tag{39}$$
$$V_1 = V_0 \cdot 26 + (\text{value of 'Z'}) = 0 \cdot 26 + 26 = 26 \tag{40}$$
$$V_2 = V_1 \cdot 26 + (\text{value of 'Z'}) = 26 \cdot 26 + 26 = 676 + 26 = 702 \tag{41}$$

The value is verified.

6.2 Problem 2: Binary Exponentiation

Task: Calculate $3^{10} \pmod 7$ using binary exponentiation.

Execution Trace (Iterative): We write the exponent $n = 10$ in binary: $(1010)_2$. We initialize res $= 1$, base $x = 3$.

1) Iteration 1 (Bit 0 of exponent is 0):

$$n = 10 \quad (\text{binary } 1010) \tag{42}$$
$$\text{Last bit} = 0 \implies \text{res remains } 1 \tag{43}$$
$$x \leftarrow (3 \cdot 3) \pmod 7 = 9 \pmod 7 = 2 \tag{44}$$
$$n \leftarrow 10 \gg 1 = 5 \tag{45}$$

2) Iteration 2 (Bit 1 of exponent is 1):

$$n = 5 \quad (\text{binary } 101) \tag{46}$$
$$\text{Last bit} = 1 \implies \text{res} \leftarrow (1 \cdot 2) \pmod 7 = 2 \tag{47}$$
$$x \leftarrow (2 \cdot 2) \pmod 7 = 4 \tag{48}$$
$$n \leftarrow 5 \gg 1 = 2 \tag{49}$$

3) Iteration 3 (Bit 2 of exponent is 0):

$$n = 2 \quad \text{(binary 10)} \tag{50}$$
$$\text{Last bit} = 0 \implies \text{res remains } 2 \tag{51}$$
$$x \leftarrow (4 \cdot 4) \pmod 7 = 16 \pmod 7 = 2 \tag{52}$$
$$n \leftarrow 2 \gg 1 = 1 \tag{53}$$

4) Iteration 4 (Bit 3 of exponent is 1):

$$n = 1 \quad \text{(binary 1)} \tag{54}$$
$$\text{Last bit} = 1 \implies \text{res} \leftarrow (2 \cdot 2) \pmod 7 = 4 \tag{55}$$
$$x \leftarrow (2 \cdot 2) \pmod 7 = 4 \tag{56}$$
$$n \leftarrow 1 \gg 1 = 0 \tag{57}$$

The loop terminates since $n = 0$. The final answer is 4.

We verify this result: $3^{10} = 59049$. Dividing by 7: $59049 = 7 \cdot 8435 + 4 \implies 59049 \equiv 4 \pmod 7$. The result is verified.

6.3 Problem 3: Container With Most Water

Task: Find the maximum water container area in the array $A = [1, 5, 4, 3]$.

Execution Trace: We initialize left pointer $i = 0$, right pointer $j = 3$, and maxArea $= 0$.

1) Step 1:

$$i = 0, \quad j = 3 \tag{58}$$
$$A[i] = 1, \quad A[j] = 3 \tag{59}$$
$$\text{width} = 3 - 0 = 3 \tag{60}$$
$$\text{height} = \min(1, 3) = 1 \tag{61}$$
$$\text{area} = 3 \cdot 1 = 3 \tag{62}$$
$$\text{maxArea} \leftarrow \max(0, 3) = 3 \tag{63}$$

Since $A[i] \leq A[j]$, we increment $i \leftarrow 1$.

2) Step 2:

$$i = 1, \quad j = 3 \tag{64}$$
$$A[i] = 5, \quad A[j] = 3 \tag{65}$$
$$\text{width} = 3 - 1 = 2 \tag{66}$$
$$\text{height} = \min(5, 3) = 3 \tag{67}$$
$$\text{area} = 2 \cdot 3 = 6 \tag{68}$$
$$\text{maxArea} \leftarrow \max(3, 6) = 6 \tag{69}$$

Since $A[i] > A[j]$, we decrement $j \leftarrow 2$.

3) Step 3:

$$i = 1, \quad j = 2 \tag{70}$$
$$A[i] = 5, \quad A[j] = 4 \tag{71}$$
$$\text{width} = 2 - 1 = 1 \tag{72}$$
$$\text{height} = \min(5, 4) = 4 \tag{73}$$
$$\text{area} = 1 \cdot 4 = 4 \tag{74}$$
$$\text{maxArea} \leftarrow \max(6, 4) = 6 \tag{75}$$

Since $A[i] > A[j]$, we decrement $j \leftarrow 1$. The loop terminates since $i = j = 1$. The final maximum area is 6.

6.4 Problem 4: Three Sum

Task: Find all unique triplets summing to zero in the array $A = [-1, 0, 1, 2, -1, -4]$.

Execution Trace:

1) Sorting Step: We sort the array: $A = [-4, -1, -1, 0, 1, 2]$. Length $N = 6$.
2) Outer Loop $i = 0$ ($A[i] = -4$): We initialize $left = 1$ ($A[left] = -1$), $right = 5$ ($A[right] = 2$).
   - Step 1: sum $= -4 + (-1) + 2 = -3 < 0 \implies left \leftarrow 2$ ($A[left] = -1$).
   - Step 2: sum $= -4 + (-1) + 2 = -3 < 0 \implies left \leftarrow 3$ ($A[left] = 0$).

- Step 3: sum $= -4 + 0 + 2 = -2 < 0 \implies left \leftarrow 4$ ($A[left] = 1$).
- Step 4: sum $= -4 + 1 + 2 = -1 < 0 \implies left \leftarrow 5$. Loop terminates since $left = right = 5$.

3) Outer Loop $i = 1$ ($A[i] = -1$): We initialize $left = 2$ ($A[left] = -1$), $right = 5$ ($A[right] = 2$).

- Step 1: sum $= -1 + (-1) + 2 = 0$. We record the triplet: $[-1, -1, 2]$. We update pointers: $left \leftarrow 3$, $right \leftarrow 4$. We check for duplicates: $A[left] = A[3] = 0 \neq A[2] \implies$ no skip. $A[right] = A[4] = 1 \neq A[5] \implies$ no skip.
- Step 2: Now $left = 3$ ($A[left] = 0$), $right = 4$ ($A[right] = 1$). sum $= -1 + 0 + 1 = 0$. We record the triplet: $[-1, 0, 1]$. We update pointers: $left \leftarrow 4$, $right \leftarrow 3$. Loop terminates since $left > right$.

4) Outer Loop $i = 2$ ($A[i] = -1$): Since $i > 0$ and $A[2] == A[1] == -1$, we execute 'continue' to avoid duplicates.
5) Outer Loop $i = 3$ ($A[i] = 0$): We initialize $left = 4$ ($A[left] = 1$), $right = 5$ ($A[right] = 2$). sum $= 0 + 1 + 2 = 3 > 0 \implies right \leftarrow 4$. Loop terminates since $left = right = 4$.
6) The outer loop completes since $i < N - 2 = 4$. The final list of unique triplets is: $[[-1, -1, 2], [-1, 0, 1]]$.

7 Conclusion

This paper has presented a formal, mathematically grounded treatment of base conversion arithmetic, binary exponentiation, and two-pointer search strategies. We demonstrated how positional number systems without zero elements (bijective systems) can be modeled recursively, and how logarithmic exponentiation processes can be implemented using binary bit manipulation.

Furthermore, we proved that the greedy pointer convergence algorithm for finding the container with the most water reduces the time complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$ by systematically pruning suboptimal states. Finally, we formalized duplicate-avoidance invariants for sorted multi-pointer arrays. These algorithms represent essential optimization paradigms for engineering high-performance software systems.